Automated Software Testing

03. Web testing with Playwright



DE HOGESCHOOL MET HET NETWERK

Elfde-Liniestraat 24, 3500 Hasselt
www.pxl.be

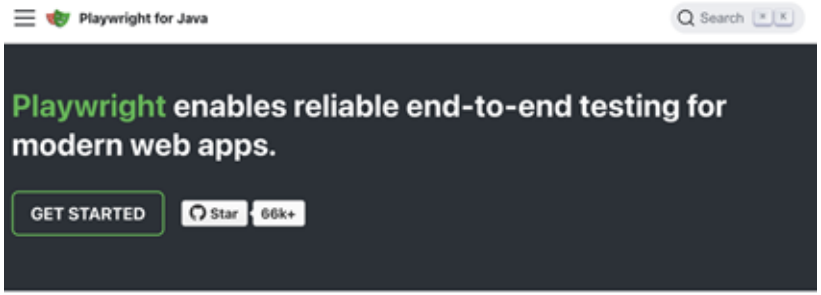
The student

- **Understands Key Concepts in Playwright**:
  - Define browser, browser context, page, and locators.
- **Masters Locators and Element Interaction**
  - Use different locator strategie to interact with elements.
- **Can Implement Assertions Using Locator Assertions**
  - Validate element states (visibility, text, enabled/disabled) using built-in assertions.
- **Can Develop and Run Test Scenarios in Playwright**
  - Design and execute automated test cases covering end-to-end user flows.



Playwright

https://playwright.dev/java/







Why Playwright?

- **Built to avoid flaky tests**
  - **Auto-waiting:** actions wait until the element is visible, stable and enabled.
  - **Retrying assertions:** assertThat(locator) keeps checking until it passes or times out.
  - **Isolation:** a fresh BrowserContext per test is fast and clean.
- **Batteries included:** Chromium, Firefox and WebKit are downloaded for you; codegen, inspector, trace viewer and screenshots built in.
- **One API** for Java, JavaScript/TypeScript, Python and .NET.

Same test: Selenium

```
driver.get(url + "catalog.html");
driver.findElement(By.cssSelector("[data-testid='filter-text']"))
      .sendKeys("moon");
driver.findElement(By.cssSelector("[data-testid='filter-button']"))
      .click();
new WebDriverWait(driver, Duration.ofSeconds(5)).until(
      ExpectedConditions.numberOfElementsToBe(
            By.cssSelector("tbody tr"), 1));
assertEquals("To the Moon and Back", driver
      .findElement(By.cssSelector("[data-testid='event-name-2']"))
      .getDomProperty("textContent"));
```

You must add the explicit wait yourself. Note: getText() returns the rendered text, getDomProperty("textContent") the DOM text.

Same test: Playwright

```
page.navigate(url + "catalog.html");
page.getByTestId("filter-text").fill("moon");
page.getByTestId("filter-button").click();


assertThat(page.locator("tbody tr")).hasCount(1);
assertThat(page.getByTestId("event-name-2"))
      .hasText("To the Moon and Back");
```

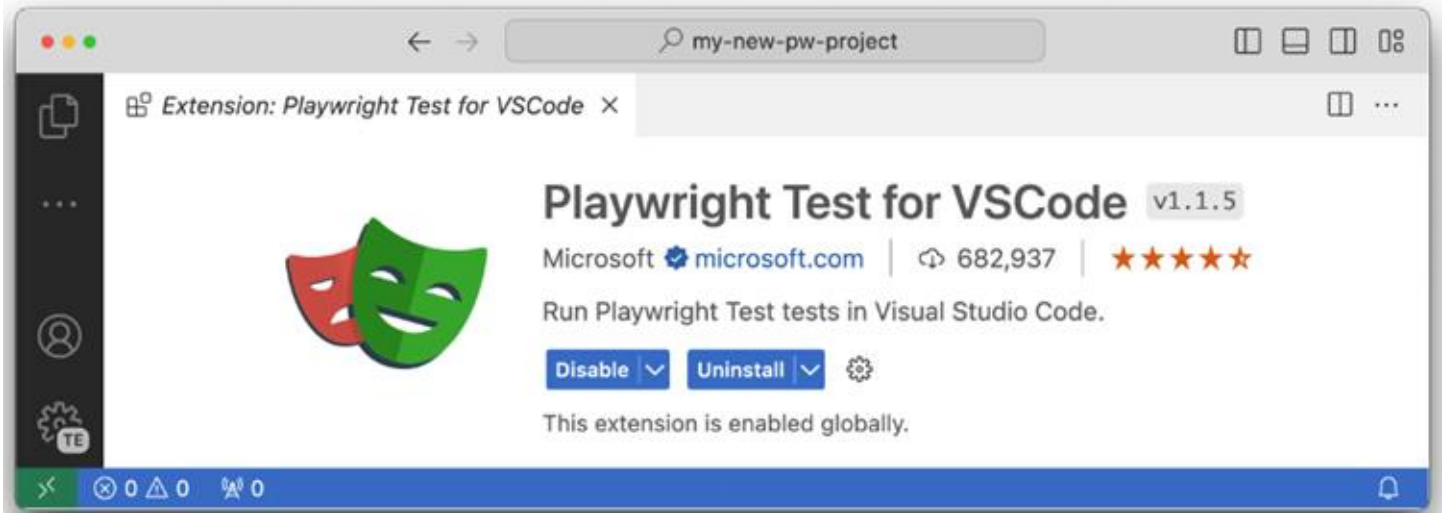
No waits: locators and assertThat retry automatically. Fewer lines, less noise.

Playwright or Selenium?

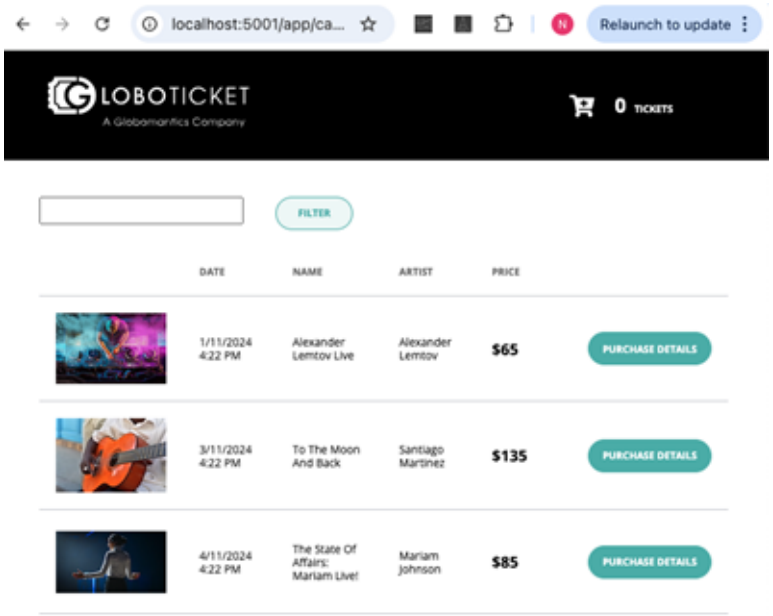
- **Choose Playwright when**
  - you start a new project or test a modern web application.
  - you want stable tests, fast feedback and strong debugging tools.
- **Selenium still makes sense when**
  - you maintain a large existing suite.
  - you need real Safari, older browsers or real devices (cloud grids).
- **Be aware:** Playwright WebKit is not real Safari.

IDE

Playwright support in IntelliJ IDEA is provided by the Test Automation ↗ plugin, and most of the features described in this section rely on it. However, basic functionality like running and debugging Playwright tests works without the plugin.



SUT


localhost:5001/app/ca...
Relaunch to update
LOBOTICKET
A Globomantics Company
0 TICKETS
FILTER
DATE
NAME
ARTIST
PRICE
1/11/2024 4:22 PM
Alexander Lemtov Live
Alexander Lemtov
$65
PURCHASE DETAILS
3/11/2024 4:22 PM
To The Moon And Back
Santiago Martinez
$135
PURCHASE DETAILS
4/11/2024 4:22 PM
The State Of Affairs: Mariam Live!
Mariam Johnson
$85
PURCHASE DETAILS

Core concepts



Playwright

```
try (Playwright playwright = Playwright.create()) {
    ...
}
```

- the entry point for all Playwright operations.
- try-with-resources - autocloseable

Browser

- create browser instances using the playwright instance.
- different **BrowserTypes**: Chromium, Firefox, or WebKit.

```
Browser browser = playwright.webkit().launch();

Browser browser = playwright.chromium().launch();

Browser browser = playwright.firefox().launch();
```

Browser

- launch a browser in **headless mode** (default) or with a UI

```
BrowserType.LaunchOptions options = new
      BrowserType.LaunchOptions().setHeadless(false);
browser = playwright.chromium().launch(options);
```

BrowserContext

- Represents an isolated browser session (like a new incognito window).
- Each BrowserContext is independent, meaning cookies, cache, and session storage do not leak between contexts.
- You can have multiple browser contexts running in a single browser instance.

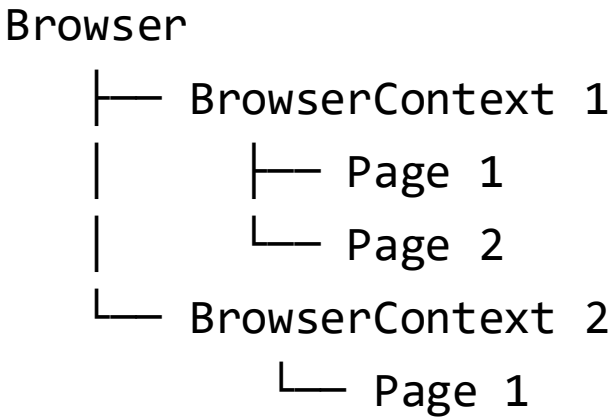
```
BrowserContext context = browser.newContext();
```

Page

- Each BrowserContext can have multiple pages.
- A Page refers to a single tab or a popup window within a browser context.
- It is used to navigate to URLs and interact with the page content.

```
Page page = context.newPage();
page.navigate("https://example.com");
```

Benefits of multiple Browser Contexts and Pages



+ **Faster Parallel Testing**: Isolated contexts allow for parallel execution without interference.
+ **Reduced Overhead**: No need to launch a new browser instance for every test case.
+ **Simulation of Multiple Users**: Each context can simulate a different user session.

Maven dependencies

```
<dependency>
    <groupId>com.microsoft.playwright</groupId>
    <artifactId>playwright</artifactId>
    <version>1.63.0</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.11.4</version>
    <scope>test</scope>
</dependency>
```

First example

```
1. @Test
2. public void firstTest() {
3.     try (Playwright pw = Playwright.create()) {
4.         Browser browser = pw.chromium().launch();
5.         Page page = browser.newPage();
6.         page.navigate("url_goes_here");
7.         System.out.println("Title: " + page.title());
8.     }
9. }
```

Locators

- Locators are the central piece of Playwright's auto-waiting and retry-ability. In a nutshell, locators represent a way to find element(s) on the page at any moment. A locator can be created with the page.locator() method.

Locators

These are the recommended built-in locators.

- Page.getByRole() to locate by explicit and implicit accessibility attributes.
- Page.getByText() to locate by text content.
- Page.getByLabel() to locate a form control by associated label's text.
- Page.getByPlaceholder() to locate an input by placeholder.
- Page.getByAltText() to locate an element, usually image, by its text alternative.
- Page.getByTitle() to locate an element by its title attribute.
- Page.getByTestId() to locate an element based on its `data-testid` attribute (other attributes can be configured).

data-testid

- A custom HTML attribute designed for easier targeting of elements in automated tests.
- Invisible to end users but accessible to testing frameworks like Jest, Cypress, Selenium,...
- Why?
  - **Test Stability:** Avoids relying on CSS classes, IDs, or text that may change with UI updates.
  - **Separation of Concerns:** Keeps test selectors separate from actual user-facing content.
  - **Improved Readability:** Makes test code clearer by providing meaningful and consistent element selectors.

data-testid

Common Issues Without `data-testid`:

- Flaky tests: Test scripts often break when UI elements like classes, IDs, or visible text are modified for styling or localization.
- Complex selectors: Tests become hard to maintain when selectors rely on deep DOM stru



Locators examples

```
page.getByLabel("User Name").fill("John");

page.getByLabel("Password").fill("secret-password");

page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in"))
    .click();

assertThat(page.getByText("Welcome, John!")).isVisible();
```

https://playwright.dev/java/docs/locators

Task 1



Write a playwright test to navigate to the <<URL>> page, and print out the inner text of all the elements with the class event-name.

Actions

Playwright can interact with HTML Input elements such as text inputs, checkboxes, radio buttons, select options, mouse clicks, type characters, keys and shortcuts as well as upload files and focus elements.

```
page.locator("a").first().click();
```

```
page.getByLabel("Birth date").fill("2020-02-02");
```

```
page.locator("input[type='checkbox']").check();
```

```
page.locator("select").selectOption("option-value");
```

https://playwright.dev/java/docs/input

LocatorAssertions

The LocatorAssertions class provides assertion methods that can be used to make assertions about the Locator state in the tests.

```
assertThat(page.title()).isEqualTo("Home");

assertThat(page.locator(".some-class").first()).isVisible();

assertThat(page.locator("p").first()).hasCSS("font-size", "16px");
```

https://playwright.dev/java/docs/test-assertions
https://playwright.dev/java/docs/api/class-locatorassertions

Auto-waiting and retrying assertions

- Before every action Playwright checks the element is attached, visible, stable and enabled.
- **assertThat(locator)** retries until it passes or times out (default 5 seconds).
- **Plain JUnit assertions do not retry:** assertEquals(1, locator.count()) can fail for an element that shows up 100 ms later.
- Prefer **assertThat(...)** over reading a value and asserting it yourself.
- Avoid **Thread.sleep** and **page.waitForTimeout**.

Taking screenshots

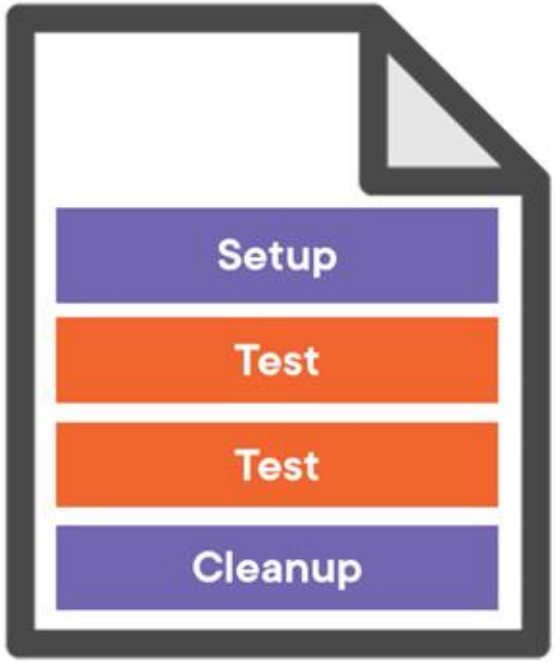
When writing tests, you might want to take screenshots of the page at different points in the test.

This can be useful for debugging, or to visually verify that the page looks as expected. This is especially useful when running end-to-end tests in a CI/CD pipeline, to ensure that a webpage looks as expected.

Playwright allows you to take screenshots of the page, elements, or the browser window.

```
page.screenshot(new Page.ScreenshotOptions().setPath("screenshot.png"));
```

Test class layout



JUnit 5

@BeforeAll
@BeforeEach
@AfterEach
@AfterAll

Generating tests

Playwright comes with the ability to generate tests out of the box and is a great way to quickly get started with testing.

It will open two windows, a browser window where you interact with the website you wish to test and the Playwright Inspector window where you can record your tests, copy the tests, clear your tests as well as change the language of your tests.

Debugging: inspector and trace viewer

- **Inspector:** run with PWDEBUG=1, or add page.pause() to a test.
- **Trace viewer:** record a trace and replay every step with DOM snapshot, network and console.
  - context.tracing().start(new Tracing.StartOptions().setScreenshots(true).setSnapshots(true));
  - context.tracing().stop(new Tracing.StopOptions().setPath(Paths.get("trace.zip")));
  - mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="show-trace trace.zip"

Task 2



1. Write a playwright test to navigate to the <<URL>> page,
2. Fill in the text “moon” in the filter field and click “Filter”
3. Verify there is only one event in the event-list.

Task 3



1. Write a playwright test to navigate to the <<URL>> page,
2. Click “Purchase details” for an event.
3. Verify the details page is displaying the right event name.
4. Change the quantity to “3” and click “Place order”
5. Verify the basket is showing the right quantity, total and number of tickets.

Task 4



1. Write a playwright test to navigate to the <<URL>> page,
2. Click “Purchase details” for an event.
3. Change the quantity to “3” and click “Place order”
4. Change the quantity to “4” and click update.
5. Verify the basket is showing the right total and number of tickets.

Task 5: bug hunt



1. Write a playwright test that filters the catalog on “Moon” (capital M) and verifies there is one event.
2. Run the test. It fails, but the same test with “moon” passes.
3. Describe the bug (steps to reproduce and expected vs actual behaviour).

Next week

- Best practices
- Using environment variables
- Page object models